

Концепция развития UI-пайплайна (Figma -> Unity) v2.0

Цель обновления

Перевести инструмент импорта из статуса «MVP для быстрого прототипирования» в статус «**Production-ready**». Основная задача — исключить потерю данных и логики на стороне Unity при внесении правок в дизайн, автоматизировать работу со шрифтами и оптимизировать хранение графических ассетов.

Текущий подход (полное удаление Canvas и пересборка с нуля) заменяется на **точечную синхронизацию, глубокую работу со стилями и переиспользование общих ресурсов.**

1. Неразрушающее обновление (Layout Updating)

Проблема: Сейчас при любом изменении в макете (сдвиг кнопки, смена цвета) плагин удаляет весь экран в Unity. Это стирает все привязанные разработчиками скрипты, анимации и звуки.

Решение: Плагин учится «узнавать» элементы и обновлять только их свойства (Transform, Color, Text), не трогая структуру и логику Unity.

Что это значит для разработчиков (Unity):

- **Компонент-метка:** При первой сборке на каждый UI-элемент вешается специальный скрипт-метка (например, `FigmaElement.cs`), который хранит уникальный ID этого элемента из Figma (`node_id`).
- **Безопасный синк:** При повторном импорте плагин не делает `Destroy()`. Он ищет объекты с соответствующими ID в текущем префабе/сцене.
- **Защита логики:** Если элемент найден, обновляются только его визуальные параметры. Любые пользовательские компоненты (`ButtonLogic`, `AudioSource`, кастомные аниматоры) остаются нетронутыми.
- **Удаленные элементы:** Если дизайнер удалил кнопку в Figma, плагин **не удаляет** ее в Unity автоматически (чтобы не сломать ссылки в коде). Он помечает ее (например, выключает `GameObject` и добавляет префикс `[DELETED]`), оставляя финальное решение за программистом.

Что это значит для дизайнеров (Figma):

- **ID — это святое:** Плагин привязывается к внутренним ID слоев в Figma.
Важное правило: если нужно изменить кнопку, меняйте ее свойства (размер, цвет). Не нужно удалять старую кнопку и рисовать на ее месте новую — иначе в Unity это распознается как удаление старого объекта со скриптами и создание нового «пустого».

2. Общие ресурсы (Shared Assets)

Проблема: Сейчас каждое окно скачивает свои собственные картинки. Если одна и та же кнопка «Закрыть» используется в 10 окнах, в Unity появится 10 дубликатов текстуры, что раздувает билд и бьет по памяти.

Решение: Внедрение папки Shared Assets.

 **Как это работает в Unity:**

- В настройках плагина появляется поле **Shared Assets Path** (например, Assets/UI/Shared/).
- Перед тем как скачать спрайт из Figma, плагин проверяет, нет ли файла с таким же именем в папке Shared.
- Если файл есть, плагин просто назначает этот спрайт на Image. Если нет — скачивает в локальную папку текущего окна.

 **Правила для дизайнеров:**

- **Единый нейминг:** Иконки, фоны и базовые элементы, которые используются сквозняком по всей игре, должны называться в Figma одинаково (например, icon_close, btn_bg_green).
- **Синхронизация с командой:** Если вы создали новый универсальный элемент, убедитесь, что он выгружен/перемещен в общую библиотеку (Shared), чтобы Unity-разработчики знали о нем.

3. Функция рескина (Reskin / Theme Duplicate)

Проблема: Создание похожих окон (например, «Магазин брони» и «Магазин оружия» или смена визуальной темы на Halloween) требует повторной настройки всей логики кнопок и скроллов в Unity.

Решение: Возможность дублировать логику с заменой арта.

 **Сценарий использования:**

1. Разработчик выбирает в Unity уже готовый и настроенный префаб окна (где висят все скрипты).
2. В плагине указывает ссылку на новый фрейм в Figma (новый скин).
3. Плагин **дублирует** префаб в новую папку.
4. Плагин проходит по структуре нового префаба и заменяет все спрайты, цвета и тексты на данные из нового Figma-фрейма (опираясь на структуру).
5. **Итог:** Готовое новое окно с другим визуалом, но полностью рабочей старой логикой. От программиста требуется только привязать окно к архитектуре игры.

4. Автоматизация типографики и стилей текста (Typography Sync)

Проблема: В текущей версии шрифты и настройки текста приходится переназначать в Unity вручную. Figma оперирует системными и веб-шрифтами, а Unity работает с собственными ассетами TextMesh Pro (TMP). Из-за этого слетают межстрочные интервалы, начертания и размеры при импорте.

Решение: Внедрение системы маппинга (сопоставления) шрифтов и автоматический пересчет параметров текста.

 **Что это значит для разработчиков (Unity):**

- **Маппинг шрифтов:** В настройках плагина появится таблица сопоставления. Разработчик один раз указывает, что шрифт Inter-Bold из Figma соответствует ассету Inter_Bold_SDF.asset в папке проекта.
- **Авто-конфигурация TMP:** Плагин не просто ставит нужный шрифт, но и автоматически переносит:
 - Font Size (размер шрифта).
 - Color (цвет текста и прозрачность).
 - Text Case (если в Figma включен All Caps, плагин включит соответствующий тумблер в TMP).
 - Alignment (выравнивание текста).
- **Конвертация Line Height:** Плагин будет математически пересчитывать межстрочный интервал из пикселей/процентов Figma в параметры Line Spacing для TextMesh Pro, чтобы верстка абзацев совпадала пиксель-в-пиксель.

Что это значит для дизайнеров (Figma):

- **Никаких «фейковых» стилей:** Крайне важно использовать реальные начертания шрифтов (выбирать Bold или SemiBold в выпадающем списке семейства), а не делать тексту обводку (Stroke), чтобы сделать его жирнее. Плагин считывает именно метаданные шрифта.
- **Ограниченный пул шрифтов:** Для UI-элементов необходимо использовать только те шрифты, которые утверждены в проекте и для которых разработчики сгенерировали SDF-ассеты в Unity. Если использовать случайный шрифт, плагин выдаст предупреждение в консоль Unity и подставит шрифт по умолчанию.
- **Чистота текстовых контейнеров:** Размеры текстового контейнера в Figma должны соответствовать логике его поведения (Auto Width, Auto Height или Fixed Size). Плагин будет транслировать эти настройки в Content Size Fitter в Unity, чтобы тексты правильно растягивались при локализации на другие языки.